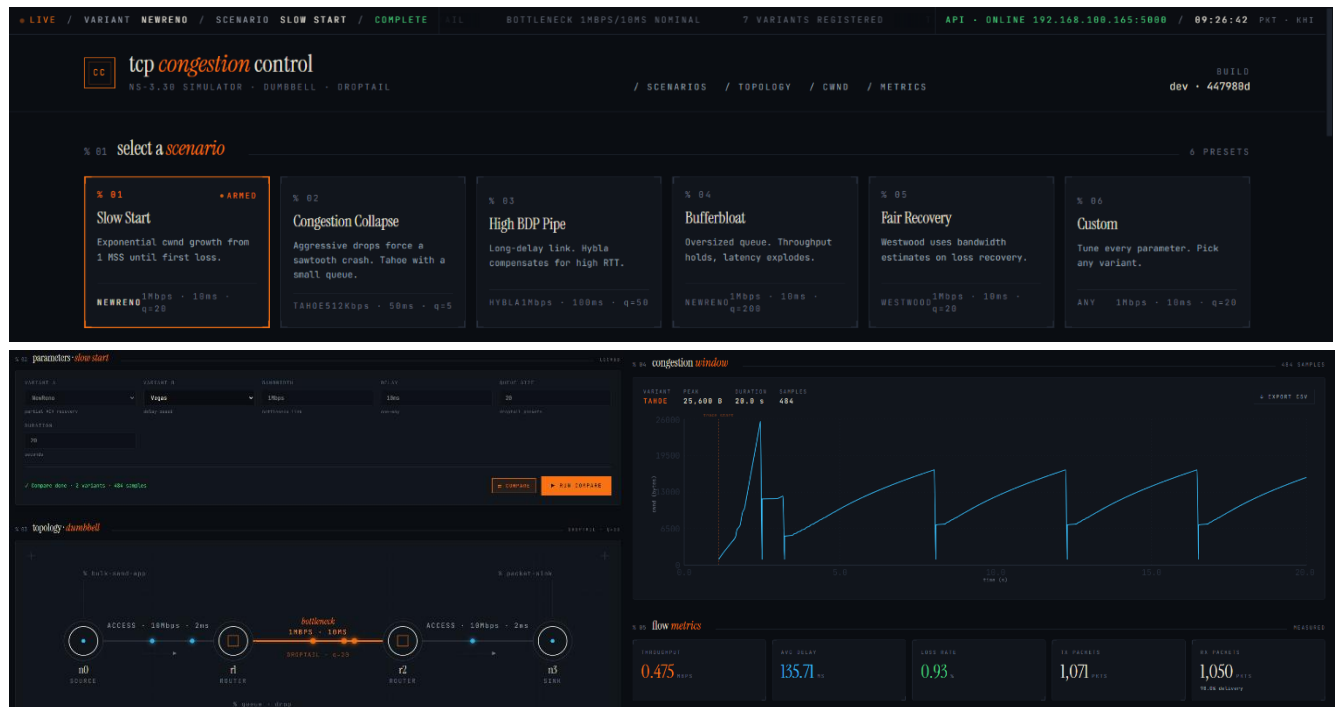




TCP Congestion Control Simulator

Computer Networks Course Project

Submitted By

Ibrahim Johar Farooqi

Project Lead

23K-0074

Syed Ukkashah Ahmed

Team Member

23K-0055

FAST School of Computing

National University of Computer & Emerging Sciences

Karachi Campus

1. Motivation

TCP congestion control is the foundational mechanism that prevents the internet from collapsing under its own load. Without it, senders would flood the network with packets, causing universal congestion collapse (Nagle, 1984). The different algorithms represent decades of research into how a sender should infer network state from indirect signals — acknowledgements, loss, and delay — and adapt its sending rate accordingly. This simulator was built to empirically observe what happens to the congestion window (cwnd) over time under different algorithms, topologies, and conditions, thereby bridging theoretical understanding to measurable, observable behaviour. The project emerges from the need to translate abstract protocol specifications into an interactive environment where the consequences of algorithmic design choices become immediately visible.

2. Overview

2.1 Significance of the Project

The congestion window (cwnd) is the core variable in all TCP variants. It limits how many bytes a sender can have in flight — sent but unacknowledged — at any time. The effective window is the minimum of cwnd and the receiver-advertised window. All congestion control algorithms are, at their core, policies for how to grow, shrink, and recover cwnd. Understanding these policies and their real-world behaviour is of direct academic and practical importance: TCP carries the overwhelming majority of internet traffic, and inefficient or poorly tuned congestion control leads to either under-utilisation of available bandwidth or congestion collapse. The simulator provides a controlled environment in which these policies can be examined, compared, and visualised. It has strong academic value for students studying network protocols, and practical value for understanding trade-offs between throughput, latency, and loss in diverse network conditions.

2.2 Description of the Project

The TCP Congestion Control Simulator is a full-stack, interactive research and teaching tool that enables simulation of seven distinct TCP congestion control variants across a configurable dumbbell network topology. Users select a TCP variant (Tahoe, Reno, NewReno, Westwood, BIC, Vegas, or Hybla), configure network parameters (bottleneck bandwidth, propagation delay, queue size, and simulation duration), and run a simulation. The system returns real-time metrics — throughput, average end-to-end delay, packet loss rate, and a time-series trace of the congestion window — and automatically generates publication-quality graphs via MATLAB.

The problem being solved is the difficulty of observing and comparing TCP behaviour without a network testbed. The scope covers seven major TCP variants spanning 1988 to 2004, a parameterised dumbbell topology, a comparison mode for side-by-side analysis, predefined pedagogical scenarios illustrating canonical phenomena (slow start, congestion collapse, bufferbloat, and high-BDP pipe filling), and integrated MATLAB graph generation for report-quality output.

2.3 Background of the Project

The project is grounded in the following foundational literature and software:

- V. Jacobson, "Congestion Avoidance and Control," ACM SIGCOMM, 1988. — Original TCP Tahoe slow start and congestion avoidance.
- W. Stevens, "TCP Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery Algorithms," RFC 2001, 1997.
- S. Floyd & T. Henderson, "The NewReno Modification to TCP's Fast Recovery Algorithm," RFC 2582, 1999. — Partial ACK handling.
- S. Mascolo et al., "TCP Westwood: Bandwidth Estimation for Enhanced Transport over Wireless Links," ACM MobiCom, 2001.
- L. Xu, K. Harfoush, & I. Rhee, "Binary Increase Congestion Control (BIC) for Fast Long-Distance Networks," IEEE INFOCOM, 2004.
- L. Brakmo & L. Peterson, "TCP Vegas: End to End Congestion Avoidance on a Global Internet," IEEE JSAC, 1995.
- C. Caini & R. Firrincieli, "TCP Hybla: A TCP Enhancement for Heterogeneous Networks," Int. Journal of Satellite Comm., 2004.
- ns-3 Network Simulator (v3.30), <https://www.nsnam.org> — discrete-event simulation engine used for all TCP variant simulations.
- MATLAB R2023b, MathWorks — used for post-simulation graph generation.
- J. Nagle, "Congestion Control in IP/TCP Internetworks," RFC 896, 1984. — Original congestion collapse problem statement.

2.4 Project Category

This project is Research-based. It does not produce a commercial product or address an industry brief; instead, it constructs a simulation and analysis environment to study, reproduce, and compare the behaviour of well-established TCP congestion control algorithms under controlled and parameterised conditions. The findings are intended for academic analysis and course evaluation at FAST-NUCES Karachi.

3. Features / Scope / Modules

The following features are listed in order of significance:

3.1 Seven TCP Variant Simulation Engine

The simulator implements all seven TCP variants — Tahoe, Reno, NewReno, Westwood, BIC, Vegas, and Hybla — using ns-3.30 as the discrete-event simulation engine. Tahoe and Reno are implemented as custom C++ subclasses of TcpNewReno, correcting for the absence of native Tahoe/Reno implementations in ns-3. The remaining five variants use ns-3 native implementations. Each variant produces a time-series cwnd trace and full FlowMonitor statistics (throughput, delay, loss).

3.2 Interactive React Dashboard

A full React + TypeScript dashboard (Vite, Tailwind CSS v4, port 5173) provides the user interface. Users configure all simulation parameters, select scenarios, toggle compare mode, and view results through recharts-powered congestion window charts. The dashboard includes animated topology visualisation with per-scenario annotations (e.g. loss events for Congestion Collapse, full-queue indicators for Bufferbloat) and progressive cwnd playback animation.

3.3 MATLAB Graph Generation Integration

After each simulation, the dashboard automatically invokes MATLAB via its command-line interface (`matlab -batch`) through a Windows Flask bridge server (port 5001). MATLAB generates four publication-quality figures — cwnd over time, throughput, average delay, and packet loss — and returns them as base64-encoded PNGs for display in the dashboard. Graphs can be downloaded as PNG files for direct inclusion in reports.

3.4 Compare Mode

A dedicated compare mode allows simultaneous execution of two TCP variants with identical network parameters. Results are displayed as overlaid cwnd traces (with per-variant colour coding), a side-by-side metrics table with best-value highlighting, and overlaid MATLAB bar charts for grouped comparison of throughput, delay, and loss.

3.5 Predefined Pedagogical Scenarios

Six predefined scenarios are provided, each demonstrating a canonical TCP phenomenon: (1) Slow Start — clean exponential cwnd growth until first loss; (2) Congestion Collapse — repeated cwnd resets under high loss; (3) High BDP Pipe — Hybla's RTT-independent growth on satellite-like paths; (4) Bufferbloat — cwnd inflation with latency spike due to oversized queues; (5) Fair Recovery — Westwood's bandwidth-estimated ssthresh vs. Reno; (6) Custom — user-defined all parameters.

3.6 CSV Export

Both single-simulation and compare-mode results can be exported as CSV files directly from the dashboard, enabling offline analysis and import into external tools such as Excel or MATLAB.

3.7 Configurable Dumbbell Topology

All simulations use a four-node dumbbell topology: sender (n0) connected via a 10 Mbps / 2 ms access link to router r1, a user-configurable bottleneck link (r1 to r2), and a 10 Mbps / 2 ms egress link from r2 to receiver n3. The bottleneck bandwidth, propagation delay, DropTail queue size, and simulation duration are all user-configurable, making the topology general enough to reproduce a wide range of real-world network conditions.

4. Project Planning

The project was divided into sequential development phases with clear deliverables and responsibility assignments.

Phase	Tasks	Responsibility	Duration
Phase 1	NS-3 environment setup; C++ simulation script (tcp-compare.cc); all 7 variant implementations; Flask API server on Ubuntu VM	Ibrahim Johar Farooqi	Weeks 1–3
Phase 2	React dashboard scaffolding; Vite + Tailwind v4 setup; API integration; cwnd chart (recharts); metrics panel	Ibrahim Johar Farooqi	Weeks 3–5
Phase 3	Topology SVG animation (Layers 1–3); scenario picker; compare mode; CSV export; design system	Both members	Weeks 5–7
Phase 4	MATLAB integration: matlab_server.py (Flask, port 5001); plot_cwnd.m; base64 PNG pipeline; dashboard MATLAB section	Syed Ukkashah	Weeks 7–9
Phase 5	End-to-end testing; bug fixes; documentation; report writing; Vercel deployment (pending)	Both members	Weeks 9–10

Responsibility was divided such that the ns-3 simulation backend, VM infrastructure, and React dashboard UI were led by Ibrahim Johar Farooqi, while the MATLAB integration, component architecture, and compare mode were led by Syed Ukkashah, with both members contributing to integration, testing, and debugging across the full stack.

5. Project Feasibility

5.1 Technical Feasibility

The project is fully technically feasible and has been verified as working end-to-end. The core simulation engine is ns-3.30, a mature and widely used open-source network simulator with native support for five of the seven TCP variants required. The remaining two (Tahoe and Reno) were implemented as custom C++ subclasses. The React + TypeScript dashboard uses stable, well-supported libraries (Vite, Tailwind CSS v4, recharts). MATLAB integration uses the standard -batch CLI mode, which is supported on all MATLAB versions from R2019b onwards. The three-tier architecture — React dashboard on Windows, ns-3 Flask server on Ubuntu VM, MATLAB Flask bridge on Windows — has been fully implemented and verified.

Technical risks that were encountered and resolved include: Python 3.6 constraints on the Ubuntu VM (resolved by using subprocess.PIPE in place of capture_output); ns-3's absence of native Tahoe and Reno implementations (resolved via custom C++ subclasses); MATLAB

cold-start latency of 15–30 seconds per invocation (accepted as a known constraint of the matlab -batch approach); and VM IP address instability under DHCP (resolved by assigning a static IP).

5.2 Economic Feasibility

The project incurs no financial cost. All primary software components are free: ns-3 (open-source, BSD licence), React and its ecosystem (open-source, MIT licence), Python and Flask (open-source), and VMware Workstation (available via university licence or Player free tier). MATLAB is available via the university's institutional licence. Development hardware consists of personal laptops and the university's computing infrastructure. There are no operational costs for the deployed version (Vercel free tier for React hosting). The benefit-to-cost ratio is very high: a fully functional network simulation and analysis platform at zero direct cost.

5.3 Schedule Feasibility

The project was completed within the ten-week semester timeline. The phased development plan (Section 4) was adhered to, with the ns-3 backend completed by Week 3, the React dashboard by Week 7, and MATLAB integration by Week 9. The remaining tasks — minor UI cosmetic fixes, Vercel deployment, and report finalisation — fall within the final week. No single phase constituted an unmanageable workload because the backend (ns-3 + Flask) and frontend (React) were developed in parallel by different team members before integration.

6. Hardware and Software Requirements

6.1 Hardware Requirements

Component	Specification
Development Machine (Windows)	Windows 10/11, Node.js 22, Python 3.x, MATLAB R2019b+, VMware Workstation
Ubuntu Virtual Machine	Ubuntu 18.04, 2 vCPUs, 4 GB RAM, gcc-8, Python 3.6, static IP 192.168.100.165, ns-3.30
Network	Host-only or bridged VMware adapter; VM reachable from host at 192.168.100.165:5000

6.2 Software Requirements

Software	Version	Purpose
ns-3	3.30	TCP simulation engine
GCC	8 (Ubuntu)	ns-3 C++ compiler
Python (VM)	3.6	Flask API server (server.py)

Flask	Latest	REST API on VM and Windows MATLAB bridge
Node.js	22	React dashboard build environment
Vite + React + TypeScript	Latest	Dashboard framework
Tailwind CSS	v4 (@tailwindcss/vite)	Dashboard styling
recharts	Latest	cwnd time-series chart
MATLAB	R2019b or later	Publication-quality graph generation
VMware Workstation / Player	Any recent	Ubuntu VM hosting
GitHub	—	Version control (syedukkashah/tcp_congestion_control)

7. Diagrammatic Representation of the Overall System

The system follows a three-tier architecture across two physical machines (Windows host and Ubuntu VM). The diagram below illustrates all components and data flows.

Windows Host Machine React Dashboard (localhost:5173) <i>Vite + TypeScript + Tailwind v4</i> <i>recharts cwnd chart + metrics panel</i> <i>Topology SVG + playback animation</i> <i>Compare mode + CSV export</i> matlab_server.py (localhost:5001) <i>Flask MATLAB bridge (Windows Python)</i> <i>Invokes: matlab -batch plot_cwnd.m</i> <i>Returns 4 PNGs as base64 to dashboard</i> analysis/plot_cwnd.m (MATLAB script) <i>Reads analysis/job.json</i> <i>Generates: cwnd.png, throughput.png, delay.png, loss.png</i>	Ubuntu VM (192.168.100.165) server.py (Flask :5000) <i>GET /api/health</i> <i>POST /api/simulate</i> <i>POST /api/compare</i> <i>GET /api/export-csv (single + compare)</i> tcp-compare.cc (ns-3 simulation) <i>Dumbbell topology (n0-r1-r2-n3)</i> <i>7 TCP variants (custom + native)</i> <i>cwnd trace via ConnectCwndTrace</i> <i>FlowMonitor: throughput, delay, loss</i>
---	---

Figure 1. Three-tier system architecture: React dashboard (Windows) communicates with ns-3 Flask server (Ubuntu VM) via HTTP on port 5000, and with the MATLAB bridge server (Windows) on port 5001.

Data Flow (Single Simulation):

1. User configures parameters in dashboard and clicks Run.
2. React POSTs to <http://192.168.100.165:5000/api/simulate> with variant, bandwidth, delay, queueSize, and duration.
3. server.py on the VM builds the ns-3 command and spawns the subprocess.
4. tcp-compare.cc runs the simulation, writes the cwnd trace to stdout and FlowMonitor XML to disk.
5. server.py parses output and returns JSON: { cwnd: [{time, cwnd}], metrics: {throughputMbps, avgDelayMs, lossRate, txPackets, rxPackets} }.
6. React renders the cwnd chart and metrics panel; simultaneously POSTs simulation data to localhost:5001/api/generate-graphs.
7. matlab_server.py writes job.json to analysis/, clears figures/, and runs: matlab - batch plot_cwnd.m.
8. plot_cwnd.m reads job.json and generates four PNGs to analysis/figures/.
9. matlab_server.py base64-encodes the PNGs and returns them to the dashboard.
10. Dashboard displays all four MATLAB-generated graphs in the Analysis section, with per-image PNG download buttons.

Dumbbell Network Topology:

n0 — [10 Mbps / 2 ms] — r1 — [BOTTLENECK (configurable)] — r2 — [10 Mbps / 2 ms] — n3

Default bottleneck: 1 Mbps bandwidth, 10 ms propagation delay, DropTail queue size 20 packets. All interesting congestion behaviour occurs at the r1-r2 bottleneck link.

8. References

- [1] V. Jacobson, "Congestion Avoidance and Control," ACM SIGCOMM Computer Communication Review, vol. 18, no. 4, pp. 314–329, 1988.
- [2] W. R. Stevens, "TCP Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery Algorithms," RFC 2001, Internet Engineering Task Force, 1997.
- [3] S. Floyd and T. Henderson, "The NewReno Modification to TCP's Fast Recovery Algorithm," RFC 2582, Internet Engineering Task Force, 1999.
- [4] S. Mascolo, C. Casetti, M. Gerla, M. Y. Sanadidi, and R. Wang, "TCP Westwood: Bandwidth Estimation for Enhanced Transport over Wireless Links," Proceedings of ACM MobiCom, Rome, Italy, 2001.

- [5] L. Xu, K. Harfoush, and I. Rhee, "Binary Increase Congestion Control (BIC) for Fast Long-Distance Networks," Proceedings of IEEE INFOCOM, Hong Kong, 2004.
- [6] L. S. Brakmo and L. L. Peterson, "TCP Vegas: End to End Congestion Avoidance on a Global Internet," IEEE Journal on Selected Areas in Communications, vol. 13, no. 8, pp. 1465–1480, 1995.
- [7] C. Caini and R. Firrincieli, "TCP Hybla: A TCP Enhancement for Heterogeneous Networks," International Journal of Satellite Communications and Networking, vol. 22, no. 5, pp. 547–566, 2004.
- [8] J. Nagle, "Congestion Control in IP/TCP Internetworks," RFC 896, Internet Engineering Task Force, 1984.
- [9] ns-3 Consortium, ns-3 Network Simulator, Version 3.30, <https://www.nsnam.org>, 2020.
- [10] MathWorks, MATLAB, Version R2023b, The MathWorks Inc., Natick, Massachusetts, 2023.
- [11] GitHub Repository: syedukkashah/tcp_congestion_control, https://github.com/syedukkashah/tcp_congestion_control